

并行整体刚度矩阵组装：二阶段组装效率与硬件适配评估

面向 mentor 下一步意见的实验整理

Jiang Haohua

2026-05-14

这次先把效率问题和 Intel 平台优先级分清楚

效率问题

- 单次组装和多次组装要分开看。
- 有符号组装和无符号直接组装要分开比。
- 重点不是换一个名字，而是看稀疏结构能不能复用。

硬件问题

- Intel 是优先展示平台，Apple 是复现和对照平台。
- 线程数增加不等于一定加速。
- Intel 的 P/E-core 用 taskset 做实际 affinity restriction。
- Apple 的 QoS profile 只能作为调度偏置实验。

实验平台先固定，后面每个数字才可比较

共享数据结构

- Mesh: 节点、单元和 DOF 映射。
- CsrMatrix: 全局刚度矩阵的稀疏结构和值。
- AssemblyPlan: 每个单元的 DOF 和写入位置。
- $\text{relative_l2} \leq 1\text{e-}8$: 数值正确性检查。

参与比较的 **CPU** 算法

- `cpu_serial`
- `cpu_atomic`
- `cpu_lock_guard`
- `cpu_private_csr`
- `cpu_coo_sort_reduce`
- `cpu_graph_coloring`
- `cpu_row_owner`

Source: README.md; docs/cpu; benchmark and symbolic evaluation source files

符号阶段和数值阶段各做一半工作

符号阶段

- 全局矩阵的 CSR sparsity pattern。
- 每个单元的全局 DOF 列表。
- 每个 $K_e(i, j)$ 对应的 CSR value index。

数值阶段

- 读取坐标和材料参数。
- 对 Tet4 计算体积、形函数梯度和 B^TDB 。
- 按 scatter 位置把 K_e 累加到全局矩阵。

这里的边界

网格拓扑和 DOF 编号不变时，符号结果可以保留；材料状态或迭代状态变化时，只重复数值阶段。

mentor 示例和当前 C++ 实现可以逐项对上

Mentor 示例概念	当前 C++ 实现	在项目里的作用
build_symbolic_pattern	CsrMatrix::build_sparsity	生成 CSR pattern
cellDofsCache	AssemblyPlan::dofs	缓存每个单元的全局 DOF
allocate_global_matrix	CsrMatrix 清零并复用 values	结构不变，只更新数值
assemble_numeric	cpu_serial 等 assembler	计算 Ke 并写回全局矩阵
无直接对应项	AssemblyPlan::scatter	提前保存 CSR 写入位置
section / closure	固定每节点 3 DOF	当前先不引入高阶 DOF 抽象

Source: docs/cpu/symbolic_numeric_assembly.md; results/2026-05-11-symbolic-numeric/symbolic_numeric_eval_report.md

效率表和硬件图来自两类平台数据

固定算例

- nodes = **228384**
- elements = **1113684**
- dofs = **685152**
- element = Tet4
- kernel = physics_tet4

平台边界

- 符号/无符号效率表: Apple M4 Max, macOS arm64, Clang 21.0.0, OpenMP 202011, physical/logical = 14/14。
- 硬件适配主平台: Intel(R) Core(TM) Ultra 7 265KF, Linux x86_64, GCC 13.3, OpenMP 201511, physical/logical = 20/20。
- Apple 时间不替代 Intel 绝对性能; Intel 图用于回答主要用户平台上的线程和核心 profile 问题。

这样做的原因

固定网格、单元类型和核函数后, 先比较符号结构复用的算法收益; 再用 Intel 实测图解释主要 CPU 平台上的硬件适配边界。

单次组装也受益，多次组装收益更明显

组装次数	符号总耗时 ms	数值/次 ms	符号复用摊销总耗时 ms	直接生成/次 ms	直接排序归并/次 ms	无符号直接总耗时 ms	收益
1	3200.968	617.818	3818.786	770.372	5505.827	6276.199	1.644x
3	3195.067	568.143	1633.165	593.735	5228.634	5822.369	3.565x
10	3256.337	575.691	901.325	549.004	5147.784	5696.788	6.320x
30	3252.907	578.315	686.745	516.919	5035.935	5552.854	8.086x

符号复用的优势来自两部分：少做结构分析，少做直接贡献的全局排序归并。 Source:

results/2026-05-11-symbolic-numeric/symbolic_numeric_eval_report.md; Apple M4 Max/macOS arm64 source run

单次和多次组装对应两种使用场景

单次组装

- 符号复用路径: **3818.786 ms**
- 无符号直接路径: **6276.199 ms**
- 主要差距来自直接路径的排序归并。

多次组装

- 3 次: **3.565x**
- 10 次: **6.320x**
- 30 次: **8.086x**
- 更贴近 Newton iteration 或 time stepping。

对项目有用的结论

如果网格和 DOF 编号不变，组装流程应该尽量保留符号结构，把后续工作压到数值阶段。

有符号和无符号的区别在写入路径

有符号组装

- CSR pattern 先固定。
- 单元 DOF 先缓存。
- $K_e(i, j)$ 写到哪里先算好。
- 数值阶段直接按位置累加。

无符号直接组装

- 每次生成所有 (row, col, value)。
- 每次对贡献做排序和 reduce。
- WindHub 单次排序归并约 **5.5 s**。
- transient memory 约 **2.39 GiB**。

比较的实质

这里比较的是“是否提前知道全局稀疏写入位置”，不是比较两个函数名。

控制实验说明：重建符号结构不是推荐做法

组装次数	符号构建次数	CSR 构建 ms	scatter plan ms	符号总耗时 ms	数值/次 ms	摊销总耗时 ms	相对无符号收益
1	1	1714.238	1505.008	3219.246	594.023	3813.269	1.646x
3	3	1711.323	1435.719	3147.042	588.176	3735.218	1.559x
10	10	1661.970	1483.861	3145.832	598.086	3743.918	1.522x
30	30	1575.358	1306.344	2881.703	512.877	3394.580	1.636x

`symbolic_rebuild_serial` 用来排除一个误解：二阶段组装的价值不在“每次都拆成两步”，而在“符号结果可以复用”。

simplified 和 physics_tet4 解决不同问题

Mesh topology

- tet4 / Abaqus C3D4
- hex8 / Abaqus C3D8
- 决定单元节点数与 DOF mapping。

Kernel mode

- simplified: synthetic local matrix。
- physics_tet4: Tet4 物理刚度。
- 非 Tet4 使用 physics_tet4 时会回退。

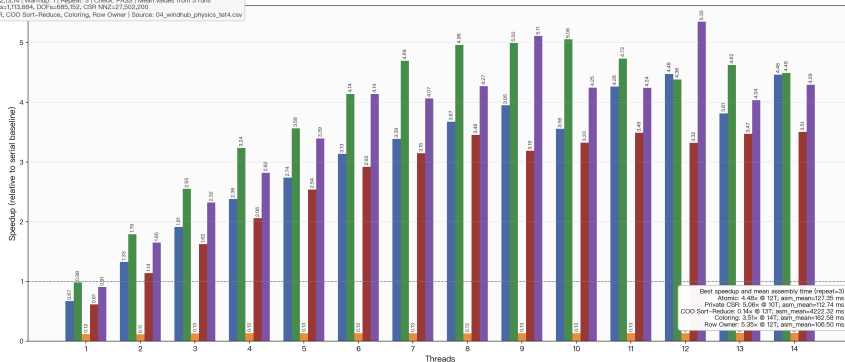
讲结果时的边界

simplified 看 assembly overhead; physics_tet4 看更接近工程 Tet4/C3D4 的组装成本。两类图不要互相替代。

真实 Tet4 物理组装下，最快点出现在 row-owner 路线

效率统计 / Efficiency Grouped Bars: 3d-WindTurbineHub / physics_tet4

配置 / Configuration
Case: 3d-WindTurbineHub | Mesh: 3d-WindTurbineHub | Element: Tet4 | Kernel: physics_tet4
Threads: 1,2,3,4,5,6,7,8,9,10,11,12,13,14 | Warmup: 1 | Repeat: 3 | Check: PASS | Mean values from 3 runs
Solver: nodes=228,384, elements=1,113,884, DOFs=685,152, CSR NNZ=27,502,200
Algorithms: Atomic, Private CSR, COO Sort-Reduce, Coloring, Row Owner | Source: 04_windhub_physics_tet4.csv



Grouped bars replace line-only presentation. Every bar is annotated with speedup to two decimals; mean assembly time is valid because repeat=3.

Atomic Private CSR COO Sort-Reduce Coloring Row Owner

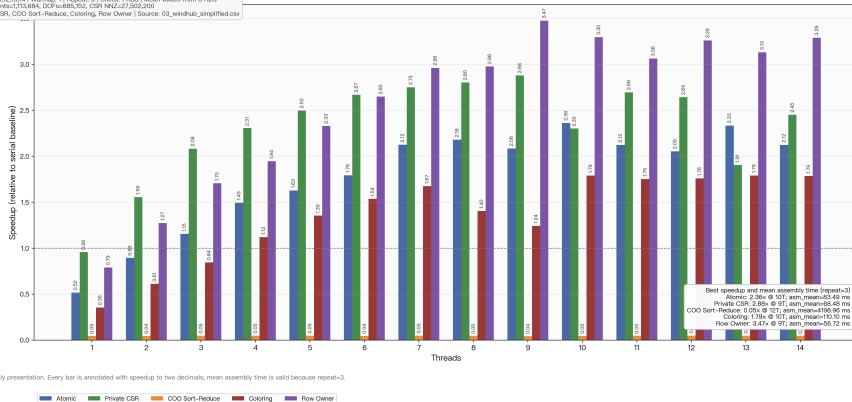
Source: 2026-04-28

repeat=3, threads=1..14, presentation_charts_12_v2

simplified 用来单独看全局组装开销

效率统计 / Efficiency Grouped Bars: 3d-WindTurbineHub / simplified

配置 / Configuration
Case: 3d-WindTurbineHub | Mesh: 3d-WindTurbineHub | Element: Tet4 | Kernel: simplified
Threads: 1,2,3,4,5,6,7,8,9,10,11,12,13,14 | Warmup: 1 | Repeat: 3 | Check: PASS | Mean values from 3 runs
Solver: nodes=228,384, elements=1,113,684, DOFs=685,152, CSR NNZ=27,502,200
Algorithms: Atomic, Private CSR, COO Sort-Reduce, Coloring, Row Owner | Source: 03_windhub_simplified.csv



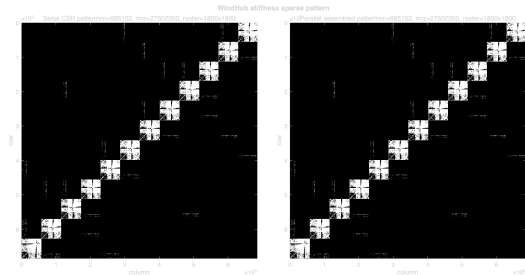
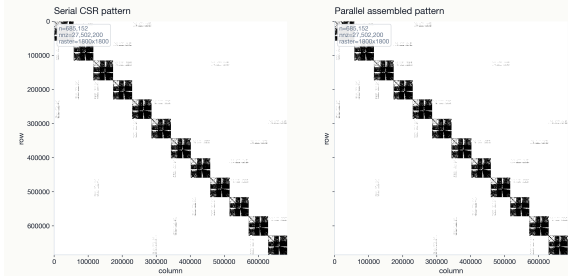
Source: 2026-04-28

repeat=3, threads=1..14, presentation_charts_12_v2

新增 sparse pattern: serial 和 parallel 结构一致

WindHub stiffness sparse pattern

3d-WindTurbineHub | physics_tet4 | cpu_atomic @ 14T | rel_l2=1.489e-16, max_abs=6.836e-03



Source: results/2026-05-16-mentor-action-items/sparse_pattern; Python and MATLAB generated from exported row,col pattern

硬件适配先从 Intel 主平台讲起

Intel Core Ultra 7 265KF

- Linux x86_64, GCC 13.3, OpenMP 201511。
- physical/logical = 20/20。
- full_host: 全 CPU 资源。
- performance_core_only: taskset restricted CPUs 0-7。
- efficiency_core_only: taskset restricted CPUs 8-19。

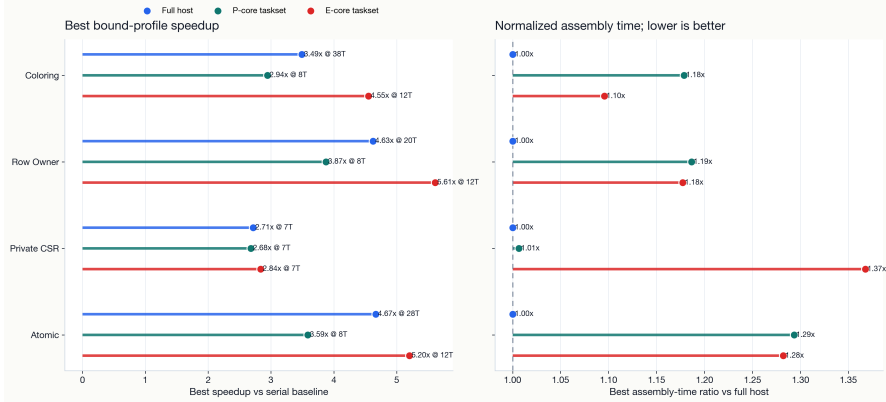
Apple M4 Max 对照

- macOS arm64, Clang 21.0.0, OpenMP 202011。
- physical/logical = 14/14。
- performance / efficiency 是 QoS-biased profile。
- Apple 没有和 Linux taskset 等价的公开硬绑核接口。

Intel: taskset 下的 P/E-core 资源限制

Intel Core Ultra 7 265KF Core-Profile Acceleration Comparison

full host vs P-core-only vs E-core-only; Linux taskset affinity-restricted profiles

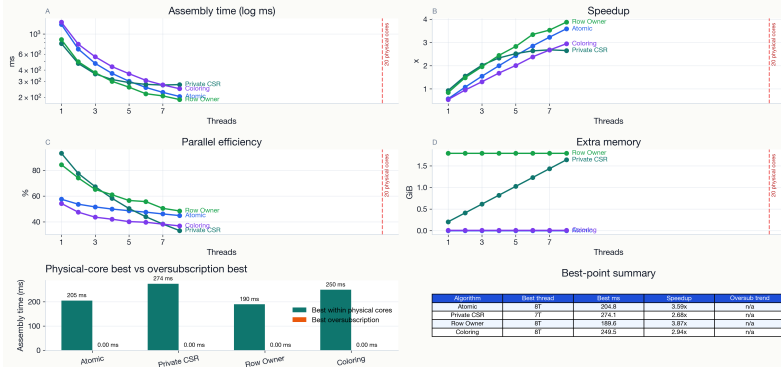


cross-platform-v1/figures; Linux taskset affinity-restricted P/E-core profiles

Intel P-core-only: taskset restricted CPUs 0-7

Thread Scaling Dashboard: bound

Large panels show scaling behavior; the lower panels summarize the fastest assembly points.



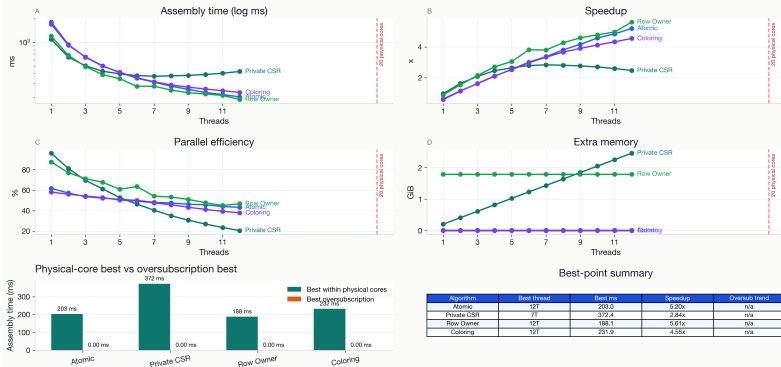
Source:

2026-05-12-thread-scaling-linux-intel-pcore

Intel E-core-only: taskset restricted CPUs 8-19

Thread Scaling Dashboard: bound

Large panels show scaling behavior; the lower panels summarize the fastest assembly points.



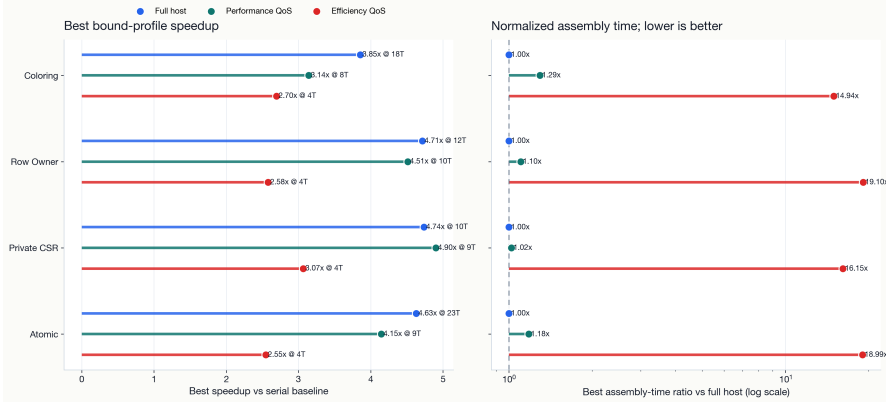
Source:

2026-05-12-thread-scaling-linux-intel-ecore

Apple: QoS 只能说明调度偏置，不能当硬绑核

Apple M4 Max Core-Profile Acceleration Comparison

full host vs Performance QoS vs Efficiency QoS; QoS-biased, not hard-pinned core affinity

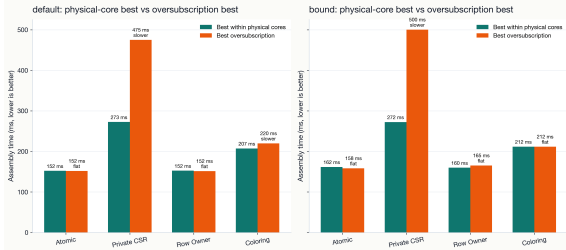


cross-platform-v1/figures; macOS QoS-biased sensitivity profiles, not hard-pinned affinity

超过物理核后的线程数是过量订阅，不是 SMT 收益

Physical Cores vs Oversubscription

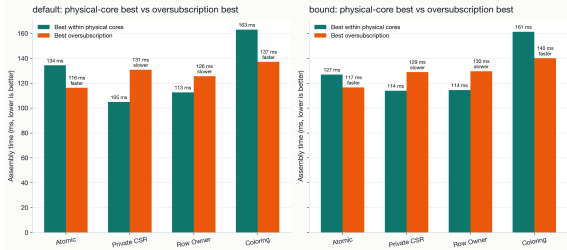
Lower bars mean faster assembly; trend labels compare oversubscription to the physical-core best.



Intel full host

Physical Cores vs Oversubscription

Lower bars mean faster assembly; trend labels compare oversubscription to the physical-core best.



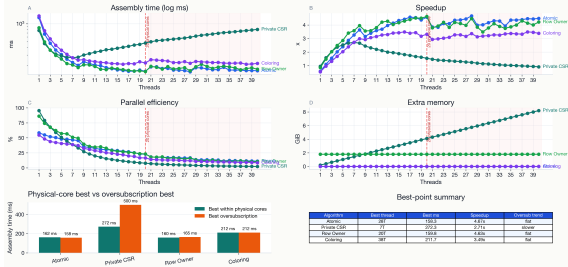
Apple M4 Max full host

当前记录中 Intel 和 Apple 都是 `physical_cores == logical_cores`；超过物理核后的线程数只能解释为 **软件线程过量订阅**。

跨平台图只用于说明边界，不做绝对排名

Thread Scaling Dashboard: bound

Large panels show scaling behavior; the lower panels summarize the fastest assembly points.

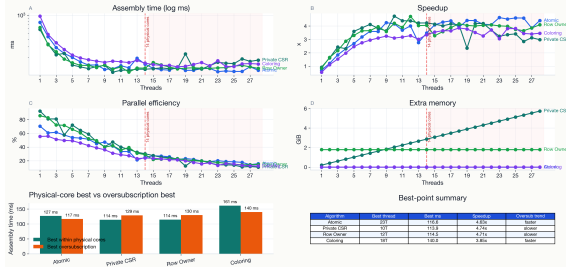


Intel full host

这两张图放在一起，是为了说明实验矩阵和趋势；硬件、OS、compiler、OpenMP runtime 和 affinity 控制都不同，不能直接写成纯算法排名。

Thread Scaling Dashboard: bound

Large panels show scaling behavior; the lower panels summarize the fastest assembly points.



Apple M4 Max full host

现在可以给 mentor 的阶段性答复

已经有数据支撑的部分

- 单次和多次组装：符号复用收益明确。
- 有符号和无符号：排序归并是直接路径的主要成本。
- 并行 symbolic：WindHub 1..14 线程 sweep 已生成。
- cpu_lock_guard：per-entry mutex baseline 已与 atomic 同口径比较。
- sparse pattern：serial/parallel row,col、MatrixMarket、Python/MATLAB spy 图已生成。
- Intel 主平台：full-host、P-core-only、E-core-only 已有实测图。
- Apple 对照平台：QoS-biased 结果用于说明调度边界。
- 物理核/逻辑核：当前没有 SMT 逻辑核收益证据。

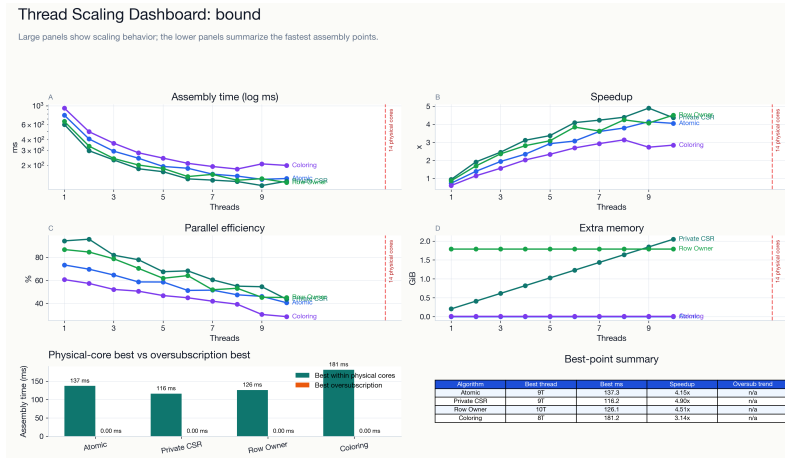
接下来值得继续做的部分

- 在 Intel 上补同口径 parallel symbolic / direct-no-symbolic 复测。
- 对 parallel symbolic 临时内存继续做峰值 RSS 交叉核验。
- 研究符号 artifact 生命周期，评估符号和数值阶段能否流水并行。
- 补充有 SMT 的 CPU，单独回答逻辑核收益。

结论：先复用结构，再谈并行扩展

- ① **二阶段组装在当前 C++ 平台里有明确实现**：CSR sparsity + scatter plan 是符号阶段，Ke 计算与 value 填充是数值阶段。
- ② **parallel symbolic 已进入可测状态**：WindHub 14 线程下 parallel_symbolic_reuse + cpu_atomic 为 **662.423 ms**，同线程 direct/no-symbolic 为 **1669.417 ms**。
- ③ **cpu_lock_guard baseline 已闭环**：per-entry mutex correctness 通过，但最快仍慢于 cpu_atomic。
- ④ **sparse pattern correctness 已补图**：serial/parallel pattern 均由项目 CSR 导出，Python 和 MATLAB spy 图已生成。
- ⑤ **下一步补齐 Intel 同口径效率表**：不要用 Apple 的 parallel symbolic / direct-no-symbolic 计时替代 Intel 绝对性能。

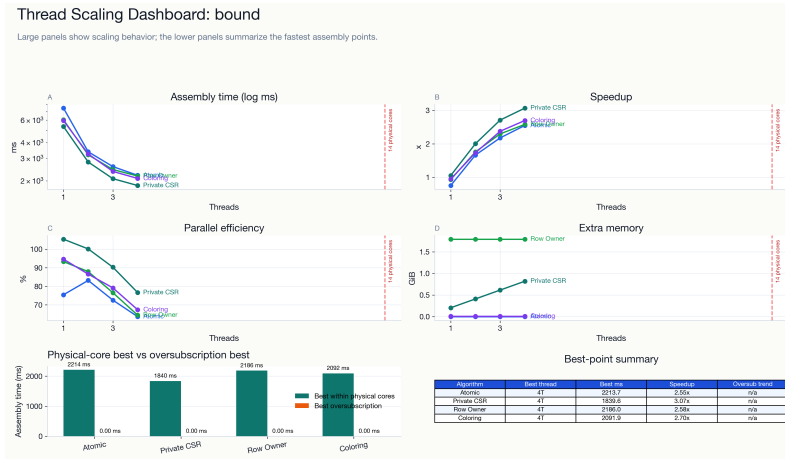
附录： Apple Performance QoS bound dashboard



Source:

2026-05-14-thread-scaling-macos-m4max-performance-qos

附录： Apple Efficiency QoS bound dashboard



Source:

2026-05-14-thread-scaling-macos-m4max-efficiency-qos

复现方式和这份材料的边界

- 这份材料包含 beamer 源码、核心图片资产和资产清单。
- 推荐使用 XeLaTeX 编译: `latexmk -xelatex -interaction=nonstopmode -halt-on-error mentor_next_steps_beamer.tex`
- 当前机器缺少 `latexmk / xelatex / pdflatex`, 所以这里只做源码、路径和引用检查。
- 所有图片均来自仓库已有结果目录; 没有新增 benchmark, 也没有新生成数据图。